

SQL Subqueries

05 April 2026 20:26

A **subquery** (also known as an **Inner Query** or **Nested Query**) is a SQL query nested inside another query. The outer query uses the results of the inner query to restrict its own data.

Think of it as a mathematical expression like $x = (2 + 3) \times 10$. You must solve the part in the parentheses first before you can solve the rest.

1. How it Works

1. **Execution Order:** The inner query (subquery) executes first.
2. **Passing Results:** The result of the inner query is passed to the outer query.
3. **Final Output:** The outer query executes using that result to produce the final output.

*There are Three type of subqueries

Great—practice is the best way to master correlated subqueries. Here's a **fresh dataset** and a set of **progressive questions** for you to solve.

 Dataset: Online Store

 Table 1: Customers

customer_id	name	city
1	Asha	Delhi
2	Ravi	Mumbai
3	Neha	Delhi
4	Arjun	Chennai

 Table 2: Orders

order_id	customer_id	order_amount	order_date
101	1	500	2024-01-10
102	1	700	2024-02-15
103	2	300	2024-01-20
104	3	900	2024-03-05
105	3	400	2024-03-10
106	4	200	2024-02-25

 Correlated Subquery Practice Questions

♦ Beginner Level

1.

Find customers whose **order amount is greater than their own average order amount**.

👉 Hint: Compare each order with `AVG(order_amount)` for that customer.

2.

Find customers who have placed **at least one order above 600**.

👉 Hint: Use EXISTS.

♦ Intermediate Level

3.

Find the **highest order for each customer**.

👉 Hint: Compare order amount with MAX(order_amount) per customer.

4.

List customers whose **total order amount is greater than the average total order amount of all customers**.

👉 Hint: Nested aggregation + correlated subquery.

5.

Find customers who have placed **more orders than the average number of orders per customer**.

👉 Hint: Count orders per customer vs average count.

♦ Advanced Level

6.

Find orders where the **order amount is greater than the average order amount of customers from the same city**.

👉 Hint: You'll need to connect Customers + Orders inside the subquery.

7.

Find customers whose **every order is greater than 300**.

👉 Hint: Use NOT EXISTS.

8.

Find customers who have **at least one order equal to the maximum order in the entire table**.

👉 Hint: Combine global max with correlation.

🚀 Challenge (Interview-Level)

9.

Find customers whose **latest order amount is greater than their previous order**.

👉 Hint:

- Compare order dates
- Use correlated subquery to find previous order

✅ 1. Orders greater than their own average

```
SELECT *  
FROM orders o  
WHERE order_amount > (  
  SELECT AVG(order_amount)
```

```
FROM orders
WHERE customer_id = o.customer_id
);
```

✓ 2. Customers with at least one order > 600 (EXISTS)

```
SELECT *
FROM customers c
WHERE EXISTS (
  SELECT 1
  FROM orders o
  WHERE o.customer_id = c.customer_id
  AND o.order_amount > 600
);
```

✓ 3. Highest order for each customer

```
SELECT *
FROM orders o
WHERE order_amount = (
  SELECT MAX(order_amount)
  FROM orders
  WHERE customer_id = o.customer_id
);
```

✓ 4. Customers whose total > avg total of all customers

```
SELECT *
FROM customers c
WHERE (
  SELECT SUM(order_amount)
  FROM orders
  WHERE customer_id = c.customer_id
) > (
  SELECT AVG(total_amount)
  FROM (
    SELECT SUM(order_amount) AS total_amount
    FROM orders
    GROUP BY customer_id
  ) t
);
```

✓ 5. Customers with more orders than average count

```
SELECT *
FROM customers c
WHERE (
  SELECT COUNT(*)
  FROM orders
  WHERE customer_id = c.customer_id
) > (
  SELECT AVG(order_count)
```

```
FROM (  
  SELECT COUNT(*) AS order_count  
  FROM orders  
  GROUP BY customer_id  
) t  
);
```

✅ 6. Orders greater than avg of same city

```
SELECT *  
FROM orders o  
JOIN customers c ON o.customer_id = c.customer_id  
WHERE o.order_amount > (  
  SELECT AVG(o2.order_amount)  
  FROM orders o2  
  JOIN customers c2 ON o2.customer_id = c2.customer_id  
  WHERE c2.city = c.city  
);
```

✅ 7. Customers whose every order > 300 (NOT EXISTS)

```
SELECT *  
FROM customers c  
WHERE NOT EXISTS (  
  SELECT 1  
  FROM orders o  
  WHERE o.customer_id = c.customer_id  
  AND o.order_amount <= 300  
);
```

✅ 8. Customers with at least one max order

```
SELECT *  
FROM customers c  
WHERE EXISTS (  
  SELECT 1  
  FROM orders o  
  WHERE o.customer_id = c.customer_id  
  AND o.order_amount = (  
    SELECT MAX(order_amount) FROM orders  
  )  
);
```

✅ 9. Latest order > previous order (🔥 Interview Level)

```
SELECT *  
FROM orders o  
WHERE order_date = (  
  SELECT MAX(order_date)  
  FROM orders  
  WHERE customer_id = o.customer_id  
)
```

```
AND order_amount > (  
  SELECT order_amount  
  FROM orders  
  WHERE customer_id = o.customer_id  
  AND order_date < o.order_date  
  ORDER BY order_date DESC  
  LIMIT 1  
);
```